```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.CardRogue.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S23_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
            // T009 wiring: the procedural pages host owns the four frozen UI pages.
            PagesHost.EnsureInScene();
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.CardRogue.Boot
```

```
{
    /// &lt;summary&gt;
    /// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
    /// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
        }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.CardRogue.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
```

```
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
```

```
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// AdGate.cs
// AdGate.cs - S1 T008 rewarded-ad gate for CardRogue (U149 pure-IAA revenue face).
// Three frozen rewarded points, all purely optional (U090 heal law: zero interstitials,
// zero forced views - an ad point is a quiet button the player may tap, never a wall
// the table throws up): AD_RESCUE backs the near-miss rescue trio (revive / reroll /
// double), AD_BOOST_DOUBLE backs the after-deal optional double-up, AD_DEX_FILL backs
// the dex-completion fill (one missing charm slot). The 12-per-day hard cap (U149
// dual-zone: the same cap binds the domestic and the overseas compliance zone -
// switching zones never loosens it) rides a rolling 24h window in the WALL-CLOCK
// domain (X995 law: an uptime-domain window dies across sessions - the new session
// clock restarts near zero, old capsules look future-dated, the prune condition never
// fires and the cap silently becomes a lifetime meter, compliant but UX-dead; wall
// seconds keep the window truly rolling across restarts). OfferAd prunes-then-caps-
// then-records; a capped offer is a silent refusal with zero mutation outside the ad
// ledger and the felt table keeps humming untouched (E4 assertion-6: 20 runs / 345
// plays / 97 of 97 deals resolved with zero ads - no system in this game is ever
// blocked by an ad). Accepting is strictly post-offer and once-per-capsule: the boost
// grant itself lives at the consuming call site (rescue choice / deal double / dex
// fill bind in T009), so the ad gate never touches the lineup, the dex or the score -
// the zero-blocking law is structural, not just behavioral. Pure C# (no UnityEngine)
// so Editor gates drive it headless. Every number is a frozen anchor (U149 / X995 /
// kit #6) - zero re-tuning, zero RNG.
using System;
using System.Collections.Generic;
namespace Sy.CardRogue
{
    /// &lt;summary&gt;
    /// Frozen ad face (U149 revenue-model law + X995 window law + kit #6 zero-ads law):
    /// the three rewarded categories, the dual-zone daily cap and the rolling window.
    /// Zero re-tuning: S2+ content work widens flavor text, never the cap or window.
    /// &lt;/summary&gt;
    public static class AdFrozen
    {
        public const int DailyCap = 12;                 // U149: 12 offers per rolling 24h, both zones
        public const double WindowSeconds = 86400.0;    // X995: 24h rolling window, wall-clock domain
        public const bool AdsRequiredForClosure = false; // kit #6/#9: ads purely optional
        public const bool InterstitialAllowed = false;  // U090: rewarded only, zero interstitials
        // U149 dual-zone compliance face: the domestic hard top and the overseas zone
        // bind the same cap - switching zones never loosens it.
        public static readonly string[] ComplianceZones = { "domestic", "overseas" };
        public const int ZoneCapDomestic = 12;
        public const int ZoneCapOverseas = 12;
        public static readonly string[] Categories =
        {
```

```
        "AD_RESCUE",        // rewarded, optional: back one near-miss rescue choice (trio)
        "AD_BOOST_DOUBLE",  // rewarded, optional: double this deal's score after a win
        "AD_DEX_FILL"      // rewarded, optional: fill one missing charm in the dex
    };
    // Frozen offer captions (purely-optional voice, per category). The refused face
    // stays cozy: a capped offer is a quiet no, never a nag.
    public static readonly string[] OfferDetails =
    {
        "optional: watch a short video to take a rescue pick (revive / reroll / double)?",
        "optional: watch a short video to double this deal's score?",
        "optional: watch a short video to fill one missing charm in the dex?"
    };
    public static bool IsCategory(string type)
    {
        for (int i = 0; i &lt; Categories.Length; i++) if (Categories[i] == type) return true;
        return false;
    }
}
/// &lt;summary&gt;One ad slot exposure: category + wall stamp + accept flag.&lt;/summary&gt;
public sealed class AdCapsule
{
    public string Type;       // one of AdFrozen.Categories
    public double OfferedAt;  // wall-clock seconds at offer time (X995 domain)
    public bool Accepted;     // one accept per capsule (strictly post-offer opt-in)
}
/// &lt;summary&gt;
/// Ad ledger (save.adcaps data face, T009 wiring). The capsule list is the
/// rolling-window truth; lifetime counters ride along for the IAA-health face.
/// &lt;/summary&gt;
public sealed class AdLedger
{
    public readonly List&lt;AdCapsule&gt; Capsules = new List&lt;AdCapsule&gt;();
    public int OffersTotal;    // lifetime offers that passed the cap
    public int AcceptsTotal;   // lifetime accepts
}
/// &lt;summary&gt;
/// T008 rewarded-ad gate. The caller (view layer, T009) stamps the wall clock at
/// offer time; the gate owns prune / cap / record / accept discipline. It never
/// self-fires and never calls into the game systems - the zero-blocking law is
/// structural, not just behavioral.
/// &lt;/summary&gt;
public sealed class AdGate
{
    public double Time;   // logical seconds (event stamps; view binds in T009)
    public string Zone = "domestic";   // U149 compliance-zone label (cap is uniform)
    public readonly List&lt;SimEvent&gt; Events = new List&lt;SimEvent&gt;();
    public readonly AdLedger Ledger = new AdLedger();
    public void Log(string type, string detail) { Events.Add(new SimEvent(Time, type, detail)); }
    public int CountEvents(string type)
    {
```

```
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
// X995 rolling-window prune: a capsule ages out at exactly 24h (inclusive
// boundary). A rolled-back wall clock never prunes - a future-dated capsule
// stays counted, so the gate can only under-offer, never over-offer.
public int Prune(double wallNow)
{
    int removed = 0;
    for (int i = Ledger.Capsules.Count - 1; i &gt;= 0; i--)
    {
        if (wallNow - Ledger.Capsules[i].OfferedAt &gt;= AdFrozen.WindowSeconds)
        {
            Ledger.Capsules.RemoveAt(i);
            removed++;
        }
    }
    return removed;
}
// In-window capsules of one category (post-prune face; the view may hide a
// slot whose category already saturated the window).
public int CountInWindow(string type, double wallNow)
{
    int n = 0;
    for (int i = 0; i &lt; Ledger.Capsules.Count; i++)
    {
        if (Ledger.Capsules[i].Type == type
            &amp;&amp; wallNow - Ledger.Capsules[i].OfferedAt &lt; AdFrozen.WindowSeconds) n++;
    }
    return n;
}
// T008 offer face: prune -&gt; cap -&gt; record. Returns true when the slot exposure
// is allowed (a capsule lands in the rolling window), false when the U149 cap
// or an unknown category refuses it. Refusal is a silent no-op: zero mutation
// outside this ledger, zero events on the game's core streams, and the caller
// simply does not show the button - the felt table hums on untouched.
public bool OfferAd(string type, double wallNow)
{
    Prune(wallNow);
    if (!AdFrozen.IsCategory(type)) return false;   // zero interstitials, zero stray types
    if (Ledger.Capsules.Count &gt;= AdFrozen.DailyCap)
    {
        Log("ad_cap_refused", "daily cap reached: the felt table hums along untouched");
        return false;
    }
    Ledger.Capsules.Add(new AdCapsule { Type = type, OfferedAt = wallNow });
    Ledger.OffersTotal++;
    int cat = Array.IndexOf(AdFrozen.Categories, type);
    Log("ad_offer", AdFrozen.OfferDetails[cat]);
```

```
            return true;
        }
        // T008 accept face: strictly post-offer opt-in, once per capsule. No open,
        // un-accepted, in-window capsule of this type = zero accepts, zero mutation
        // (an empty ledger, a capped-out face, an aged-out capsule or a second tap on
        // the same capsule all refuse). The boost grant lives at the consuming call
        // site - this face only writes the accept bookkeeping.
        public bool AcceptOffer(string type, double wallNow)
        {
            if (!AdFrozen.IsCategory(type)) return false;
            AdCapsule best = null;
            for (int i = 0; i &lt; Ledger.Capsules.Count; i++)
            {
                AdCapsule c = Ledger.Capsules[i];
                if (c.Type != type || c.Accepted) continue;
                if (wallNow - c.OfferedAt &gt;= AdFrozen.WindowSeconds) continue;  // aged out: gone
                if (best == null || c.OfferedAt &gt; best.OfferedAt) best = c;      // newest open slot
            }
            if (best == null) return false;
            best.Accepted = true;
            Ledger.AcceptsTotal++;
            Log("ad_accept", type);
            return true;
        }
    }
}
// AudioRuntime.cs
// AudioRuntime.cs - S1 T010 audio wiring face for CardRogue: two procedural seats
// (X027 zero-prefab law - the BGM loop seat plus the one-shot SFX seat) and the
// real-path hook drain. Null-safe clip lookup: the TJ production window has not
// delivered the AU-01/03/04/05/08 clips yet, so playback stays silent while the
// hook counters always advance (SY_MOCK-style: wire on arrival, count from day
// one). BGM seat law: loop enabled + playOnAwake off + the X1026 smpl1024
// loop-point alignment constant (AudioLoopFix -Mode bgm re-injects the loop
// metadata at sample 1024 when the clip lands). Editor-gate safe: one-shots
// are count-only outside play mode, so the batchmode AudioCheck drives the same
// real paths headless with deterministic counts.
using System.Collections.Generic;
using UnityEngine;
namespace Sy.CardRogue
{
    public static class AudioRuntime
    {
        // X1026: the BGM loop point lands at sample 1024 (head-pad + loop marker
        // re-injected by the arrival tooling; the seat declares and honors it).
        public const int BgmLoopSampleAlign = 1024;
        static readonly Dictionary&lt;string, int&gt; _hooks = new Dictionary&lt;string, int&gt;();
        static readonly Dictionary&lt;string, AudioClip&gt; _clips = new Dictionary&lt;string, AudioClip&gt;();
        static AudioSource _bgmSeat;
        static AudioSource _sfxSeat;
```

```
static bool _bgmArmed;
public static bool BgmArmed { get { return _bgmArmed; } }
public static AudioSource BgmSeat { get { EnsureSeats(); return _bgmSeat; } }
public static AudioSource SfxSeat { get { EnsureSeats(); return _sfxSeat; } }
public static void ResetCounts()
{
    _hooks.Clear();
    _bgmArmed = false;
}
public static int Count(string stem)
{
    int v;
    return _hooks.TryGetValue(stem, out v) ? v : 0;
}
static void Bump(string stem)
{
    int v;
    _hooks.TryGetValue(stem, out v);
    _hooks[stem] = v + 1;
}
static void EnsureSeats()
{
    if (_bgmSeat != null) return;
    var go = new GameObject("AudioRuntimeRoot");
    // static (non-MonoBehaviour) class: Object statics need explicit qualification
    if (Application.isPlaying) UnityEngine.Object.DontDestroyOnLoad(go);
    _bgmSeat = go.AddComponent&lt;AudioSource&gt;();
    _bgmSeat.loop = true;              // X1026 BGM loop seat
    _bgmSeat.playOnAwake = false;
    _sfxSeat = go.AddComponent&lt;AudioSource&gt;();
    _sfxSeat.playOnAwake = false;
}
/// &lt;summary&gt;Editor-gate cleanup face: destroy the procedural seat root
/// (edit-mode batchmode checks must leave the scene untouched).&lt;/summary&gt;
public static void TeardownSeats()
{
    if (_bgmSeat != null)
    {
        Object.DestroyImmediate(_bgmSeat.gameObject);
        _bgmSeat = null;
        _sfxSeat = null;
    }
    _clips.Clear();
}
static AudioClip ClipFor(string stem)
{
    AudioClip c;
    if (_clips.TryGetValue(stem, out c)) return c;
    c = Resources.Load&lt;AudioClip&gt;("Audio/" + stem);   // null-safe: assets pending
    _clips[stem] = c;
```

```
            return c;
        }
        /// &lt;summary&gt;BGM loop seat arm (AU-01, S01 global face): counts the seat
        /// beat, configures the loop seat and plays when the clip exists. With no
        /// clip yet the seat stays armed and silent - zero crash, zero tofu.&lt;/summary&gt;
        public static void PlayBgm()
        {
            _bgmArmed = true;
            Bump(AudioStems.BgmCardParlor);
            EnsureSeats();
            AudioClip c = ClipFor(AudioStems.BgmCardParlor);
            if (c == null) return;                 // silent until the TJ window delivers AU-01
            _bgmSeat.clip = c;
            _bgmSeat.loop = true;
            _bgmSeat.Play();
        }
        /// &lt;summary&gt;One-shot stem beat (SFX seat). Counts always; plays only in
        /// play mode with a live clip - the editor gate reads the counts.&lt;/summary&gt;
        public static void PlayOneShot(string stem)
        {
            Bump(stem);
            if (!Application.isPlaying) return;     // headless count-only face
            AudioClip c = ClipFor(stem);
            if (c == null) return;                  // null-safe lookup
            EnsureSeats();
            _sfxSeat.PlayOneShot(c);
        }
        /// &lt;summary&gt;Real-path hook drain: walk the sim event stream from
        /// fromIndex (exclusive) and fire the wired stems for the success-path
        /// events. The mapping is frozen - play_tap -&gt; slap, score_pop -&gt; mult
        /// burst, charm_grant / charm_pick -&gt; pick, near_miss -&gt; sting. Rejected
        /// plays and loss faces without stems fire nothing. The caller tracks
        /// the drain index (the view host drains after every drive step).&lt;/summary&gt;
        public static void Drain(GrayboxSim sim, int fromIndex)
        {
            if (sim == null) return;
            var evs = sim.Events;
            int start = fromIndex &lt; 0 ? 0 : fromIndex;
            for (int i = start; i &lt; evs.Count; i++)
            {
                string t = evs[i].type;
                if (t == "play_tap") PlayOneShot(AudioStems.PlayCardSlap);          // AU-03 success play
                else if (t == "score_pop") PlayOneShot(AudioStems.MultBurstPop);    // AU-04 settlement burst
                else if (t == "charm_grant" || t == "charm_pick") PlayOneShot(AudioStems.CharmPickPluck); // AU-05
                else if (t == "near_miss") PlayOneShot(AudioStems.NearMissSting);   // AU-08 zero-punishment tone
            }
        }
    }
}
// AudioStems.cs
```

```
// AudioStems.cs - S1 T010 frozen audio stem registry for CardRogue (demand-doc
// section 5.1 work orders; the TJ production window delivers the clips later -
// the wiring face lands first, null-safe by construction).
// Stem law (X1004): every stem reference goes through these constants, never a
// bare literal elsewhere. Five wired stems in this batch: the BGM loop seat
// (AU-01 bgm_card_parlor, X1026 smpl1024 loop-point law), the core feedback
// trio (AU-03 play_card_slap / AU-04 mult_burst_pop / AU-05 charm_pick_pluck -
// red-line-11 same-tick hooks on the real success paths, E4 assertion 3 frozen
// at 1.50s), and the near-miss sting (AU-08, the T006 three-face linkage's
// audio beat: caption x banner x sting). The remaining six stems ride the
// asset-arrival X027 four-piece batch per section 5.3.
namespace Sy.CardRogue
{
    public static class AudioStems
    {
        public const string BgmCardParlor = "bgm_card_parlor";     // AU-01 table BGM loop
        public const string PlayCardSlap = "play_card_slap";       // AU-03 play feedback #1
        public const string MultBurstPop = "mult_burst_pop";       // AU-04 mult settlement #2
        public const string CharmPickPluck = "charm_pick_pluck";  // AU-05 charm pick #3
        public const string NearMissSting = "near_miss_sting";    // AU-08 zero-punishment sting
        public static readonly string[] Wired =
        {
            BgmCardParlor, PlayCardSlap, MultBurstPop, CharmPickPluck, NearMissSting
        };
    }
}
// EventPopupRuntime.cs
// EventPopupRuntime.cs - S2 T013 c2 event popup wiring face for CardRogue (the
// comedy event popup flow at the four in-deal beat positions: deal transition /
// near-miss / the x8 climax shot / charm pick - zero numbers, zero punishment,
// pure content, U131 tone law). Three style families (news / documentary / wuxia)
// with a deterministic round-robin rotation: the same voice never fires twice in
// a row (anti-cliche third ban), zero RNG by construction. Copy resolves through
// LocaleService by the composed key ev_&lt;beat&gt;_&lt;family&gt; - zero hardcoded copy in
// the runtime face (T010 law). AR-11 event_popup_frame consumption is null-safe
// and armed-silent before the art lands (AudioRuntime wire-on-arrival law): the
// frame seat loads from Resources/Art by stem, stays tinted per family while the
// sprite is absent, and never crashes, never shows tofu. The sim is only read:
// this runtime adds ZERO sim state, so every frozen S1 face stays byte-identical.
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.CardRogue
{
    public static class EventPopupRuntime
    {
        // ---- frozen beat ids (taskbook T013: deal transition / near-miss / climax / charm pick)
        public const string BeatDeal = "deal";
        public const string BeatNear = "near";
        public const string BeatClimax = "climax";
```

```
public const string BeatPick = "pick";
public static readonly string[] Beats = { BeatDeal, BeatNear, BeatClimax, BeatPick };
// ---- three style families (news / documentary / wuxia), rotation-enforced
public static readonly string[] Families = { "news", "doc", "wuxia" };
// one frame, three families: programmatic tint per family (frozen)
public static readonly Color32 TintNews = new Color32(0xFF, 0xD7, 0x00, 0xFF);    // broadcast gold
public static readonly Color32 TintDoc = new Color32(0x9F, 0xC6, 0xDB, 0xFF);     // lens mist blue
public static readonly Color32 TintWuxia = new Color32(0x26, 0x9B, 0x5E, 0xFF);  // rivers-and-lakes jade
public const string KeyPrefix = "ev_";
// AR-11 event_popup_frame (512x384 PNG-32, art manifest section 4): the seat
// loads from Resources/Art by stem - null-safe, armed-silent before arrival.
public const string FrameStem = "event_popup_frame";
public const string FramePath = "Art/" + FrameStem;
static readonly List&lt;string&gt; _fired = new List&lt;string&gt;();
static readonly Dictionary&lt;string, int&gt; _shown = new Dictionary&lt;string, int&gt;();
static int _rot;
static Sprite _frame;
static bool _frameTried;
public static bool FrameArmed { get; private set; }
public static Sprite FrameSprite
{
    get
    {
        if (!_frameTried)
        {
            _frameTried = true;
            _frame = Resources.Load&lt;Sprite&gt;(FramePath);   // null-safe: AR-11 pending
        }
        return _frame;
    }
}
public static void Reset()
{
    _fired.Clear();
    _shown.Clear();
    _rot = 0;
    FrameArmed = false;
}
public static int FiredCount { get { return _fired.Count; } }
public static string FiredAt(int i) { return i &gt;= 0 &amp;&amp; i &lt; _fired.Count ? _fired[i] : null; }
public static string LastShownKey { get { return _fired.Count &gt; 0 ? _fired[_fired.Count - 1] : null; } }
public static int ShownCount(string key)
{
    int v;
    return _shown.TryGetValue(key, out v) ? v : 0;
}
public static Color32 TintOf(string family)
{
    if (family == Families[1]) return TintDoc;
    if (family == Families[2]) return TintWuxia;
```

```
    return TintNews;
}
public static string BadgeOf(string family) { return family.ToUpperInvariant(); }
public static string FamilyOf(string key)
{
    if (key == null) return Families[0];
    int cut = key.LastIndexOf('_');
    return cut &gt;= 0 &amp;&amp; cut + 1 &lt; key.Length ? key.Substring(cut + 1) : Families[0];
}
/// &lt;summary&gt;Family rotation: round-robin over the three families so the same
/// voice never fires twice in a row (anti-cliche third ban). Deterministic.&lt;/summary&gt;
static string NextFamily()
{
    string f = Families[_rot % Families.Length];
    _rot++;
    return f;
}
/// &lt;summary&gt;Show one event popup: compose the key, count it, arm the frame
/// seat. Copy itself resolves in the view through LocaleService (the
/// runtime never holds display copy - zero hardcoded copy).&lt;/summary&gt;
static void Show(string beat)
{
    string key = KeyPrefix + beat + "_" + NextFamily();
    _fired.Add(key);
    int v;
    _shown.TryGetValue(key, out v);
    _shown[key] = v + 1;
    FrameArmed = true;
    Sprite s = FrameSprite;   // armed-silent load attempt (no crash pre-arrival)
}
/// &lt;summary&gt;Real-path beat drain (AudioRuntime twin): walk the sim event
/// stream from fromIndex (exclusive) and fire the popup at the four beat
/// faces - deal_fan (deal transition), near_miss, a rocket play (the x8
/// climax shot), charm_pick. The caller tracks the drain index.&lt;/summary&gt;
public static void Drain(GrayboxSim sim, int fromIndex)
{
    if (sim == null) return;
    var evs = sim.Events;
    int start = fromIndex &lt; 0 ? 0 : fromIndex;
    for (int i = start; i &lt; evs.Count; i++)
    {
        string t = evs[i].type;
        if (t == "deal_fan") Show(BeatDeal);
        else if (t == "near_miss") Show(BeatNear);
        else if (t == "charm_pick") Show(BeatPick);
        else if (t == "play_tap" &amp;&amp; evs[i].detail != null &amp;&amp;
                 evs[i].detail.IndexOf("rocket", StringComparison.Ordinal) &gt;= 0)
            Show(BeatClimax);
    }
}
```

```
    }
}
// GrayboxBot.cs
// GrayboxBot.cs - S1 T011 port of the E4 graybox kit bare-play bot (M6-B lane; kit archive
// = Design/evidence/m6_cardrogue_graybox_kit/GrayboxBot.cs, frozen at E4 authoring - zero
// re-tuning, zero RNG). Runs 20 short runs of 5 deals each on frozen deterministic deal
// templates. Most deals are won on the scripted greedy line (bombs first, then runs, then
// leftovers). Runs r%7==3 hit a hard target on deal 4 (score lands inside the 0.9x..1.0x
// near-miss band) -&gt; three rescue offers pop up, the bot declines all of them (zero ads),
// the run ends honestly, and the charm dex keeps everything (zero punishment).
// Charm picks between won deals follow the first-free-slot rule starting at
// (2 + deal + run) % 12 - deterministic, no duplicates within a run, full 12-charm dex
// coverage across the 20-run session. Frozen session baseline (E4 Stage2, kit report
// attempt3): runs=20 plays=345 deals=97 (started==resolved) deal_wins=94 deal_losses=3
// near_misses=3 rescue_offers=9 double_offers=94 run_wins=17 run_losses=3 dex=12/12
// share_cards=15 first card hex=8. Pure C# (no UnityEngine): the editor gate drives it
// headless and the view host can replay the same lines.
using System;
using Sy.CardRogue;
namespace Sy.CardRogue
{
    public sealed class GrayboxBot
    {
        private readonly GrayboxSim sim;
        public int Runs;
        public GrayboxBot(GrayboxSim sim) { this.sim = sim; }
        // Deal templates (frozen at E4 authoring): hand, scripted plays, targets.
        // Deal 1 (idx 0) target 1,000 vs scripted 1,400; deal 2 (idx 1) 2,500 vs 3,280;
        // deal 3 (idx 2) 8,000 vs 10,080; deal 4 (idx 3) 100,000 vs 138,560 normal or
        // hard 146,000 (near-miss band on r%7==3 runs); deal 5 (idx 4) climax 100,000,000
        // vs 108,544,000 with the x8192 rocket finish.
        public static readonly int[][] DealHands = new int[][]
        {
            new int[] { 3, 3, 3, 4, 5, 6, 7, 8, 9, 10 },
            new int[] { 7, 7, 8, 8, 9, 9, 10, 11 },
            new int[] { 5, 5, 5, 6, 6, 6, 7, 8, 13, 13 },
            new int[] { 9, 9, 9, 9, 3, 4, 5, 6, 7, 14 },
            new int[] { 13, 13, 13, 13, 12, 12, 12, 12, 9, 9, 9, 9, 5, 5, 5, 5, 16, 17 }
        };
        public static readonly int[][][] DealPlays = new int[][][]
        {
            new int[][] { new int[] { 3, 3, 3, 4 }, new int[] { 5, 6, 7, 8, 9 }, new int[] { 10 } },
            new int[][] { new int[] { 7, 7, 8, 8, 9, 9 }, new int[] { 10 }, new int[] { 11 } },
            new int[][] { new int[] { 5, 5, 5, 6, 6, 6 }, new int[] { 13, 13 }, new int[] { 7 }, new int[] { 8 } },
            new int[][] { new int[] { 9, 9, 9, 9 }, new int[] { 3, 4, 5, 6, 7 }, new int[] { 14 } },
            new int[][] { new int[] { 13, 13, 13, 13 }, new int[] { 12, 12, 12, 12 }, new int[] { 9, 9, 9, 9 }, new int[
        };
        public static readonly double[] DealTargets = new double[] { 1000, 2500, 8000, 100000, 100000000 };
        public const double Deal3HardTarget = 146000;   // 138,560 / 146,000 = 94.9% -&gt; near-miss band
        int FirstFreeCharm(int startIdx)
```

```
        {
            int n = GrayboxSim.CharmRoster.Length;
            for (int off = 0; off &lt; n; off++)
            {
                int cand = ((startIdx % n) + off) % n;
                bool taken = false;
                for (int i = 0; i &lt; sim.Lineup.Count; i++) if (sim.Lineup[i] == cand) { taken = true; break; }
                if (!taken) return cand;
            }
            return startIdx % n;
        }
        public void Run(int runs)
        {
            for (int r = 0; r &lt; runs; r++)
            {
                Runs++;
                sim.StartRun(r);
                bool alive = true;
                for (int d = 0; d &lt; 5 &amp;&amp; alive; d++)
                {
                    bool hard = (r % 7 == 3) &amp;&amp; (d == 3);
                    double target = hard ? Deal3HardTarget : DealTargets[d];
                    sim.StartDeal(d, target, DealHands[d].Length);
                    for (int p = 0; p &lt; DealPlays[d].Length; p++) sim.PlayCard(DealPlays[d][p]);
                    bool won = sim.ResolveDeal();
                    if (!won) { alive = false; sim.RunLost(r); }
                    else if (d &lt; 4) sim.PickCharm(FirstFreeCharm(2 + d + r));
                }
                if (alive)
                {
                    sim.RunWon();
                    if (sim.Dex.Count &gt;= 8) sim.ShareCard();   // lineup/dex card, never a score form
                }
            }
        }
    }
}
// GrayboxSim.cs
// GrayboxSim.cs - pure C# deterministic logic core for CardRogue (S1 T003 port of the E4 graybox kit frozen source, M6-
// Common-knowledge card base (doudizhu hand-type canon, 21pt-family bases BANNED per B904
// guardrail #1) x one subversion mechanic: the Joker multiplier ladder (compound mult).
// Each play scores TypeChips x TypeMult x 2^doublings. Doublings come from the charm
// lineup (the Joker build - every charm doubles), bombs played earlier in the deal, and
// the rocket's own x8 climax. Short-run IAA shape: 5 escalating deals per run, near-miss
// rescue offers (revive / reroll / double - purely optional rewarded-video points, never
// required, the loop closes with zero ads), an optional double-up offer after each won
// deal. Charm dex is the zero-punishment meta face: a lost run never wipes the dex.
// Share face (B904 guardrail #2): the share card is a charm lineup / dex achievement card
// with an X032-style code; score-form result display is kept out of the share payload.
// No UnityEngine dependency: sim time is logical seconds; wall-clock perf is measured by
```

```
// the editor check. Zero RNG by construction: rosters, templates and picks are modular.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.CardRogue
{
    public sealed class SimEvent
    {
        public double t;
        public string type;
        public string detail;
        public SimEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
    }
    public enum HandType
    {
        None = 0, Single, Pair, Triple, TripleOne, TripleTwo,
        Straight, PairStraight, Plane, PlaneWings, FourTwo, Bomb, Rocket
    }
    public sealed class GrayboxSim
    {
        // ---- Stage1 hook timeline (frozen at E4 authoring - zero loosening) ----
        public const double CaptionT = 0.15;       // hook caption (red line 11, T+0-3s)
        public const double StarterCharmT = 0.50;  // two starter charms + multiplier teach (x4)
        public const double DealFanT = 1.00;       // first deal fans out
        public const double FirstPlayT = 1.50;     // first valid play (zero-learning-cost 3s hook)
        public const double FirstPickT = 5.50;     // first deliberate charm pick (build spine beat)
        // ---- scoring table (frozen): TypeChips x TypeMult per hand type ----
        public static readonly int[] TypeChips = { 0, 5, 10, 30, 35, 40, 60, 80, 100, 120, 160, 240, 400 };
        public static readonly int[] TypeMult = { 1, 1, 2, 3, 3, 4, 4, 5, 6, 7, 8, 16, 32 };
        // ---- 12-charm roster: cute fantasy tokens, zero gambling vocabulary, zero real IP ----
        public static readonly string[] CharmRoster =
        {
            "paper_tiger_charm", "moon_rabbit_charm", "koi_lantern_charm", "cloud_whisk_charm",
            "candy_bolt_charm", "star_fan_charm", "plum_blossom_charm", "dragon_kite_charm",
            "lucky_cat_charm", "bamboo_ring_charm", "jade_bell_charm", "firecracker_star_charm"
        };
        // ---- B904 guardrail vocabulary bans (gambling-family imagery banned from every face) ----
        public static readonly string[] ForbiddenTokens =
        {
            "casino", "jackpot", "wager", "blackjack", "baccarat", "slot_machine",
            "twenty_one", "pai_gow", "chip_stack", "bet_coin", "mahjong_parlor", "dealer_button"
        };
        public static readonly string[] PunishTokens = { "punish", "penalty", "dex_reset", "charm_wipe", "score_down", "
        public static readonly string[] ShareFormTokens = { "score", "points", "result", "rank" };
        // ---- S2 school (style) face: 3 schools over the frozen 12-charm roster ----
        // OFF by default: AttunedSchool stays SchoolNone until a player calls AttuneSchool
        // (one stance per run). The E4 bot never attunes, so every frozen S1 face (truth
        // table, x8192 climax math, near-miss band, 20-run baseline) stays byte-identical.
        // Attuning resonates: playing the attuned school's signature hand type stacks one
```

```
// in-deal flip per attuned-school charm in the lineup onto the InDealDoublings axis
// (forward, same semantics as the bomb technical flip). Base mult stays exactly
// +1 per charm (uniformity preserved; zero gacha zero paid depth).
public const int SchoolNone = -1, SchoolBomb = 0, SchoolStraight = 1, SchoolSingle = 2;
public static readonly string[] SchoolNames = { "bomb_school", "straight_school", "single_school" };
public static readonly int[] SchoolOf = { 2, 2, 1, 1, 0, 0, 1, 2, 2, 1, 0, 0 }; // 4/4/4 partition, roster mappi
public static readonly HandType[][] SchoolSignature =
{
    new HandType[] { HandType.Bomb, HandType.Rocket },
    new HandType[] { HandType.Straight, HandType.PairStraight, HandType.Plane, HandType.PlaneWings },
    new HandType[] { HandType.Single }
};
public int AttunedSchool = SchoolNone;
public static int RngCalls = 0;                  // stays 0 forever: everything is deterministic
public const bool AdsRequiredForClosure = false; // ads purely optional (E3#4 analog)
public double SimTime;
public readonly List&lt;SimEvent&gt; Events = new List&lt;SimEvent&gt;();
public readonly List&lt;int&gt; Lineup = new List&lt;int&gt;();     // charm build: each charm = one doubling
public readonly HashSet&lt;int&gt; Dex = new HashSet&lt;int&gt;();   // collected charms, never wiped on loss
public int InDealDoublings;
public long DealScore;
public double TargetScore;
public long LastScore;
public long LastMult;
public int DealsStarted, DealsResolved, DealsWon, DealsLost, NearMisses;
public int PlaysTotal, RejectedPlays, RescueOffers, DoubleOffers, AdsAccepted, ShareCards;
public int RunCount, RunWins, RunLosses;
// ---- Stage3 dense state ----
public readonly int[] CardStream = new int[2000];
public readonly double[] HandHeat = new double[300];
public readonly double[] MultTicker = new double[512];
public readonly double[] GlowField = new double[1024];
public void Log(string type, string detail) { Events.Add(new SimEvent(SimTime, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
// ==== doudizhu hand-type detection (strict canon; jokers only pair as the rocket;
// straights and runs stop at Ace; no 21pt-family shape exists in this table) ====
public static HandType DetectHandType(int[] ranks)
{
    int n = ranks.Length;
    if (n == 0) return HandType.None;
```

```
int[] c = new int[18];
bool little = false, big = false;
for (int i = 0; i &lt; n; i++)
{
    int r = ranks[i];
    if (r &lt; 3 || r &gt; 17) return HandType.None;
    if (r == 16) { if (little) return HandType.None; little = true; }
    else if (r == 17) { if (big) return HandType.None; big = true; }
    else c[r]++;
}
if (little &amp;&amp; big) return (n == 2) ? HandType.Rocket : HandType.None;
if (little || big) return (n == 1) ? HandType.Single : HandType.None;
if (n == 1) return HandType.Single;
if (n == 2) return (c[ranks[0]] == 2) ? HandType.Pair : HandType.None;
if (n == 3)
{
    bool threeSame = true;
    for (int r = 3; r &lt;= 15; r++) if (c[r] != 0 &amp;&amp; c[r] != 3) threeSame = false;
    return threeSame ? HandType.Triple : HandType.None;
}
if (n == 4)
{
    bool four = false;
    for (int r = 3; r &lt;= 15; r++) if (c[r] == 4) four = true;
    if (four) return HandType.Bomb;
    for (int r = 3; r &lt;= 15; r++) if (c[r] == 3) return HandType.TripleOne;
    return HandType.None;
}
if (n == 5)
{
    int t3 = -1, t2 = -1;
    for (int r = 3; r &lt;= 15; r++) { if (c[r] == 3) t3 = r; if (c[r] == 2) t2 = r; }
    if (t3 &gt;= 0 &amp;&amp; t2 &gt;= 0) return HandType.TripleTwo;
    bool fiveSingles = true;
    for (int r = 3; r &lt;= 15; r++) if (c[r] &gt; 1) fiveSingles = false;
    if (fiveSingles)
    {
        int sMin = 99, sMax = 0;
        for (int r = 3; r &lt;= 15; r++) if (c[r] == 1) { if (r &lt; sMin) sMin = r; if (r &gt; sMax) sMax =
        if (sMax - sMin == 4 &amp;&amp; sMax &lt;= 14) return HandType.Straight;
    }
    return HandType.None;
}
// distinct rank span summary for the run families
int distinct = 0, minR = 99, maxR = 0, fourR = -1;
for (int r = 3; r &lt;= 15; r++)
{
    if (c[r] == 0) continue;
    distinct++;
    if (r &lt; minR) minR = r;
```

```
            if (r &gt; maxR) maxR = r;
            if (c[r] == 4) fourR = r;
        }
        int countVal = c[minR];
        bool allSame = true;
        for (int r = 3; r &lt;= 15; r++) if (c[r] != 0 &amp;&amp; c[r] != countVal) allSame = false;
        if (allSame &amp;&amp; countVal == 1 &amp;&amp; distinct &gt;= 5 &amp;&amp; (maxR - minR) == distinct - 1 &a
        if (allSame &amp;&amp; countVal == 2 &amp;&amp; distinct &gt;= 3 &amp;&amp; (maxR - minR) == distinct - 1 &a
        if (allSame &amp;&amp; countVal == 3 &amp;&amp; distinct &gt;= 2 &amp;&amp; (maxR - minR) == distinct - 1 &a
        // four-with-two family: 4 + pair / 4 + two singles / 4 + two pairs
        if (fourR &gt;= 0)
        {
            if (n == 6) return HandType.FourTwo;
            if (n == 8)
            {
                int pairs = 0;
                for (int r = 3; r &lt;= 15; r++) if (c[r] == 2) pairs++;
                if (pairs == 2) return HandType.FourTwo;
            }
        }
        // plane with wings (strict shape): k&gt;=2 consecutive triples + k distinct singles, or k pairs
        if (n &gt;= 8)
        {
            int k = 0, firstT = -1, lastT = -1;
            for (int r = 3; r &lt;= 15; r++) if (c[r] == 3) { k++; if (firstT &lt; 0) firstT = r; lastT = r; }
            if (k &gt;= 2 &amp;&amp; (lastT - firstT) == k - 1 &amp;&amp; lastT &lt;= 14)
            {
                int rest = n - 3 * k;
                if (rest == k)
                {
                    bool singlesOk = true;
                    for (int r = 3; r &lt;= 15; r++) if (c[r] != 0 &amp;&amp; c[r] != 3 &amp;&amp; c[r] != 1) single
                    if (singlesOk) return HandType.PlaneWings;
                }
                if (rest == 2 * k)
                {
                    int pairWings = 0;
                    for (int r = 3; r &lt;= 15; r++) if (c[r] == 2) pairWings++;
                    if (pairWings == k) return HandType.PlaneWings;
                }
            }
        }
        return HandType.None;
    }
    public static string TypeName(HandType t)
    {
        switch (t)
        {
            case HandType.Single: return "single";
            case HandType.Pair: return "pair";
```

```
        case HandType.Triple: return "triple";
        case HandType.TripleOne: return "triple_with_single";
        case HandType.TripleTwo: return "triple_with_pair";
        case HandType.Straight: return "straight";
        case HandType.PairStraight: return "pair_straight";
        case HandType.Plane: return "plane";
        case HandType.PlaneWings: return "plane_with_wings";
        case HandType.FourTwo: return "four_with_two";
        case HandType.Bomb: return "bomb";
        case HandType.Rocket: return "rocket";
        default: return "none";
    }
}
static bool HasRank(int[] ranks, int rank)
{
    for (int i = 0; i &lt; ranks.Length; i++) if (ranks[i] == rank) return true;
    return false;
}
// ==== run / deal / play API ====
public void Caption(string text) { Log("caption", text); }
public void StartRun(int runIdx)
{
    RunCount++;
    Lineup.Clear();
    InDealDoublings = 0;
    DealScore = 0;
    AttunedSchool = SchoolNone;
    Log("run_start", "run " + runIdx + ": two starter charms join the lineup");
    GrantCharm(0);
    GrantCharm(1);
    Log("mult_teach", "x4! every charm in the lineup doubles the multiplier (2 charms)");
}
public void GrantCharm(int charmIdx)
{
    Lineup.Add(charmIdx);
    Dex.Add(charmIdx);
    Log("charm_grant", CharmRoster[charmIdx] + " joins as a starter charm");
}
public void PickCharm(int charmIdx)
{
    Lineup.Add(charmIdx);
    Dex.Add(charmIdx);
    Log("charm_pick", CharmRoster[charmIdx] + " joins the lineup (double the multiplier)");
}
public void StartDeal(int dealIdx, double target, int cardCount)
{
    InDealDoublings = 0;
    DealScore = 0;
    TargetScore = target;
    DealsStarted++;
```

```
    Log("deal_fan", "deal " + (dealIdx + 1) + " fans out: " + cardCount + " cards, target " + target.ToString("0
}
// Play a combo: validate, score with the compound mult, log the same-tick feedback
// triple (tap + sfx + score pop, red line 11). Bombs double the deal mult for
// subsequent plays; a lone joker single doubles it too; the rocket carries its own
// x8 into its own score (the climax exception) and x8 forward.
public long PlayCard(int[] ranks)
{
    HandType t = DetectHandType(ranks);
    if (t == HandType.None)
    {
        RejectedPlays++;
        LastScore = -1; LastMult = -1;
        Log("play_rejected", "not a valid combo shape");
        return -1;
    }
    int ownBonus = (t == HandType.Rocket) ? 3 : 0;
    int doublings = Lineup.Count + InDealDoublings + ownBonus;
    long mult = 1L &lt;&lt; doublings;
    long score = (long)TypeChips[(int)t] * (long)TypeMult[(int)t] * mult;
    DealScore += score;
    PlaysTotal++;
    LastScore = score; LastMult = mult;
    Log("play_tap", "play a " + TypeName(t));
    Log("sfx_card", "card snap and slide");
    Log("score_pop", "score=" + score + " mult=x" + mult);
    if (t == HandType.Bomb) InDealDoublings += 1;
    else if (t == HandType.Rocket) InDealDoublings += 3;
    else if (t == HandType.Single &amp;&amp; (HasRank(ranks, 16) || HasRank(ranks, 17))) InDealDoublings += 1;
    if (AttunedSchool != SchoolNone &amp;&amp; SchoolSignatureHit(AttunedSchool, t))
        InDealDoublings += SchoolCharmCount(AttunedSchool);
    return score;
}
// S2 attunement: one free per-run player choice (never gated by ads or currency).
public void AttuneSchool(int schoolIdx)
{
    if (schoolIdx &lt; 0 || schoolIdx &gt; 2) return;
    if (AttunedSchool != SchoolNone) return;
    AttunedSchool = schoolIdx;
    Log("school_attune", "the lineup attunes to " + SchoolNames[schoolIdx]);
}
bool SchoolSignatureHit(int school, HandType t)
{
    for (int i = 0; i &lt; SchoolSignature[school].Length; i++)
        if (SchoolSignature[school][i] == t) return true;
    return false;
}
int SchoolCharmCount(int school)
{
    int n = 0;
```

```
    for (int i = 0; i &lt; Lineup.Count; i++) if (SchoolOf[Lineup[i]] == school) n++;
    return n;
}
public bool ResolveDeal()
{
    DealsResolved++;
    if (DealScore &gt;= TargetScore)
    {
        DealsWon++;
        Log("deal_won", "target reached, the deal clears");
        DoubleOffer();
        return true;
    }
    DealsLost++;
    if ((double)DealScore &gt;= TargetScore * 0.9 - 1e-9)
    {
        NearMisses++;
        Log("near_miss", "so close - the deal fell just short, rescue offers pop up");
        RescueOffer();
    }
    else
    {
        Log("deal_lost_clean", "the deal falls short, the run ends here");
    }
    return false;
}
// Near-miss rescue (C013 point density): three optional rewarded-video outs.
// The bot never accepts; the loop still closes (zero forced ads).
public void RescueOffer()
{
    Log("rescue_offer", "optional video: revive and retry this deal?");
    Log("rescue_offer", "optional video: reroll the last play?");
    Log("rescue_offer", "optional video: double the deal score to pass?");
    RescueOffers += 3;
}
public void DoubleOffer()
{
    DoubleOffers++;
    Log("double_offer", "optional video: double this deal score?");
}
public void RunWon() { RunWins++; Log("run_won", "five deals cleared - the run flips a fortune"); }
public void RunLost(int runIdx)
{
    RunLosses++;
    Log("run_lost", "run " + runIdx + " ends, charms collected stay in the dex");
}
// Share face (B904 guardrail #2): lineup/dex achievement card with an X032-style
// code. Score-form result display is kept out of the share payload by construction.
public void ShareCard()
{
```

```
    ShareCards++;
    StringBuilder sb = new StringBuilder("CR1-");
    for (int i = 0; i &lt; CharmRoster.Length; i++)
        if (Dex.Contains(i)) sb.Append(i.ToString("X", CultureInfo.InvariantCulture));
    Log("share_card", "charm lineup card code=" + sb + " dex=" + Dex.Count + "/" + CharmRoster.Length + " (show
}
// ==== Stage3 dense deal (perf probe only) ====
public void BuildDense()
{
    for (int i = 0; i &lt; 2000; i++) CardStream[i] = 3 + ((i * 7) % 15);
    for (int t = 0; t &lt; 300; t++) HandHeat[t] = ((t * 13) % 17) * 0.05;
    for (int m = 0; m &lt; 512; m++) MultTicker[m] = ((m * 11) % 19) * 0.05;
    for (int g = 0; g &lt; 1024; g++) GlowField[g] = ((g * 5) % 23) * 0.05;
}
// Dense resolve: counting pass over the 2000-card stream, candidate enumeration
// (singles/pairs/triples/bombs per rank + straight windows), best-score pick under
// a dense mult, then heat / ticker / glow smoothing passes.
public bool DenseDealResolve(int idx)
{
    int[] cnt = new int[18];
    for (int i = 0; i &lt; 2000; i++) cnt[CardStream[i] &amp; 15]++;
    long denseMult = 1L &lt;&lt; (6 + (idx % 7));
    long best = 0;
    for (int r = 3; r &lt;= 15; r++)
    {
        if (cnt[r] &gt;= 1) { long s = (long)TypeChips[1] * TypeMult[1] * denseMult; if (s &gt; best) best = s;
        if (cnt[r] &gt;= 2) { long s = (long)TypeChips[2] * TypeMult[2] * denseMult; if (s &gt; best) best = s;
        if (cnt[r] &gt;= 3) { long s = (long)TypeChips[3] * TypeMult[3] * denseMult; if (s &gt; best) best = s;
        if (cnt[r] == 4) { long s = (long)TypeChips[11] * TypeMult[11] * denseMult; if (s &gt; best) best = s; }
    }
    for (int len = 5; len &lt;= 12; len++)
    {
        for (int start = 3; start + len - 1 &lt;= 14; start++)
        {
            bool ok = true;
            for (int r = start; r &lt; start + len; r++) if (cnt[r] &lt; 1) { ok = false; break; }
            if (ok) { long s = (long)TypeChips[6] * TypeMult[6] * denseMult; if (s &gt; best) best = s; }
        }
    }
    for (int t = 0; t &lt; 300; t++) HandHeat[t] = (HandHeat[t] + (((t * 13) % 17) * 0.05)) * 0.5;
    for (int m = 0; m &lt; 512; m++) MultTicker[m] = (MultTicker[m] + MultTicker[(m * 31 + 7) % 512]) * 0.5;
    for (int g = 0; g &lt; 1024; g++) GlowField[g] = (GlowField[g] + GlowField[(g * 17 + 3) % 1024]) * 0.5;
    return best &gt; 0;
}
// Raw idle tick: stream drift + ticker relax + glow relax.
public void DenseTick()
{
    for (int i = 0; i &lt; 2000; i++) CardStream[i] = 3 + ((CardStream[i] - 3 + 1) % 15);
    for (int m = 0; m &lt; 512; m++) MultTicker[m] = (MultTicker[m] + MultTicker[(m * 31 + 7) % 512]) * 0.5;
    for (int g = 0; g &lt; 1024; g++) GlowField[g] = (GlowField[g] + GlowField[(g * 17 + 3) % 1024]) * 0.5;
```

```
        }
    }
}
// LocaleService.cs
// LocaleService.cs - S1 T010 i18n externalization face for CardRogue: the closed
// 35-key set (demand-doc section 6.3, frozen by the ConfigValidator key-set gate)
// loaded from the real cfg_i18n table with the En/zh dual face. En is the L0
// default face (U114); the view layer resolves every piece of display copy
// through Get(key) - zero hardcoded copy (the AudioCheck hardcode gate scans
// the runtime sources). A missing key falls back to the key text itself: a
// visible diagnostic, never a crash, never tofu.
using System.Collections.Generic;
using UnityEngine;
namespace Sy.CardRogue
{
    public static class LocaleService
    {
        public const string DefaultLang = "en";
        public static string Lang = DefaultLang;
        [System.Serializable] class Row { public string key; public string en; public string zh; }
        [System.Serializable] class Table { public int version; public string table; public string[] langs; public List&
        static Dictionary&lt;string, Row&gt; _map;
        static void EnsureLoaded()
        {
            if (_map != null) return;
            _map = new Dictionary&lt;string, Row&gt;();
            var asset = Resources.Load&lt;TextAsset&gt;("cfg_i18n");
            if (asset == null) return;                 // table absent: Get falls back to the key
            var t = JsonUtility.FromJson&lt;Table&gt;(asset.text);
            if (t == null || t.strings == null) return;
            for (int i = 0; i &lt; t.strings.Count; i++)
            {
                Row r = t.strings[i];
                if (r != null &amp;&amp; !string.IsNullOrEmpty(r.key) &amp;&amp; !_map.ContainsKey(r.key)) _map[r.key] =
            }
        }
        public static bool TryGet(string key, out string value)
        {
            EnsureLoaded();
            Row r;
            if (_map != null &amp;&amp; _map.TryGetValue(key, out r) &amp;&amp; r != null)
            {
                value = Lang == "zh" ? r.zh : r.en;
                return !string.IsNullOrEmpty(value);
            }
            value = key;                               // fallback face: the key itself
            return false;
        }
        public static string Get(string key)
        {
```

```
            string v;
            TryGet(key, out v);
            return v;
        }
        public static bool HasKey(string key)
        {
            EnsureLoaded();
            return _map != null &amp;&amp; _map.ContainsKey(key);
        }
        public static int KeyCount
        {
            get { EnsureLoaded(); return _map == null ? 0 : _map.Count; }
        }
    }
}
// PagesGate.cs
// PagesGate.cs - S1 T009 pages gate for CardRogue (frozen page registry + the red-line-11
// per-second teach table + the save face codec).
// Three acceptance faces from the taskbook row T009: the frozen page list (S99_Boot /
// S01_Table / S02_Codex / S03_Share - zero missing, zero extra, every canvas carries a
// U184 scaler + raycaster, built by the view host in PagesHost.cs), the eight-beat teach
// table (demand-doc section 3, frozen at E4 authoring: 0.15 hook caption / 0.50 two
// starter charms + x4 teach / 1.00 deal fan / 1.50 first-play triple feedback / 2.50
// straight 960 / 3.50 first win 1,400 / 5.50 first charm pick / 8.00 the x8192 teaching
// loop closes), and the save face (lineup / dex / deal progress / adcaps, strict
// compose-parse round-trip through ISaveService; the AdLedger save.adcaps data face
// wired here per the T008 AdGate contract). The teach drive replays the exact E4
// Stage1 timeline on the real sim: 420 (35x3x4, the x4 first feedback) at 1.50s,
// 960 (60x4x4) at 2.50s, 20 at 3.50s -&gt; 1,400 &gt;= 1,000 first win, first pick at
// 5.50s. Pure C# (no UnityEngine) so Editor gates drive it headless and the view
// host consumes the same frozen faces. Every number is a frozen anchor (E4 kit /
// section 3) - zero re-tuning, zero RNG.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using Sy.Core;
namespace Sy.CardRogue
{
    /// &lt;summary&gt;
    /// Frozen page registry (section 2 page list) and teach anchors. Zero re-tuning:
    /// S2+ widens flavor, never the page set, the slot counts or the beat times.
    /// &lt;/summary&gt;
    public static class PageFrozen
    {
        public static readonly string[] Names = { "S99_Boot", "S01_Table", "S02_Codex", "S03_Share" };
        public const int LineupSlots = 6;    // six-charm full lineup (x64 face, section 2)
        public const int CodexCells = 12;    // twelve-grid charm wall (section 2)
        public const string SaveSlot = "cardrogue_save";
        // section-3 beats that are not anchored on a GrayboxSim constant:
```

```
    public const double TeachStraightT = 2.50;  // straight 960 - the mult is visibly live
    public const double TeachFirstWinT = 3.50;  // single 20 closes 1,400 &gt;= 1,000 - first win
    public const double TeachClimaxT = 8.00;    // the x8192 teaching loop closes
    // U184 per-canvas law (view host consumes these):
    public const int RefWidth = 1080;
    public const int RefHeight = 1920;
    public static bool IsPage(string name)
    {
        for (int i = 0; i &lt; Names.Length; i++) if (Names[i] == name) return true;
        return false;
    }
}
/// &lt;summary&gt;One red-line-11 teach beat: frozen time + id + page + i18n caption key +
/// the event anchor the beat teaches. The view resolves captions through
/// LocaleService by CaptionKey (T010 wiring); CaptionMirror stays as the frozen
/// English documentation face of the key. Null caption = a pure feedback beat (the
/// anchor itself is the feedback: tap triple / score pop / win). Pure data, zero logic.&lt;/summary&gt;
public sealed class TeachBeat
{
    public readonly double T;
    public readonly string Id;
    public readonly string Page;
    public readonly string CaptionKey;
    public readonly string CaptionMirror;
    public readonly string Anchor;
    public TeachBeat(double t, string id, string page, string captionKey, string captionMirror, string anchor)
    {
        T = t; Id = id; Page = page; CaptionKey = captionKey; CaptionMirror = captionMirror; Anchor = anchor;
    }
}
public static class PagesGate
{
    // ---- the frozen eight-beat table (demand-doc section 3, E4 authoring). The five
    // sim-anchored times bind structurally onto GrayboxSim constants; the three
    // teach-only beats bind onto PageFrozen constants. Monotonic by construction.
    public static readonly TeachBeat[] TeachBeats =
    {
        new TeachBeat(GrayboxSim.CaptionT,     "hook",        "S01_Table", "caption_hook_frame", "This run: 8192x.",
        new TeachBeat(GrayboxSim.StarterCharmT,"charm_teach", "S01_Table", "teach_mult_caption", "x4! Every charm in
        new TeachBeat(GrayboxSim.DealFanT,    "deal_fan",    "S01_Table", "deal_start_hint",    "Cards dealt - the t
        new TeachBeat(GrayboxSim.FirstPlayT,  "first_play",  "S01_Table", null, null,
        new TeachBeat(PageFrozen.TeachStraightT, "straight",  "S01_Table", null, null,
        new TeachBeat(PageFrozen.TeachFirstWinT, "first_win", "S01_Table", null, null,
        new TeachBeat(GrayboxSim.FirstPickT,   "first_pick", "S01_Table", "first_pick_hint",    "Pick a charm for th
        new TeachBeat(PageFrozen.TeachClimaxT,  "climax",     "S01_Table", "climax_teach_face",  "Six charms, bombs
    };
    // ---- the E4 Stage1 teach drive, replayed on the real sim with the frozen beat
    // times (the view paces the same beats in wall time; the gate replays them in
    // logical time). 1.50s: 35x3x4 = 420 (the x4 first feedback). 2.50s: 60x4x4 =
    // 960. 3.50s: 5x1x4 = 20 -&gt; deal 1,400 &gt;= 1,000, the first win beat. 5.50s:
```

```
// the first deliberate charm pick (koi lantern, first free slot). 8.00s: the
// x8192 teaching loop closes on the climax caption.
public static void RunTeachDrive(GrayboxSim sim)
{
    sim.SimTime = GrayboxSim.CaptionT;        // 0.15 hook frame
    sim.Caption("This run: 8192x.");
    sim.SimTime = GrayboxSim.StarterCharmT;   // 0.50 starters + x4 teach
    sim.StartRun(0);
    sim.SimTime = GrayboxSim.DealFanT;        // 1.00 the first deal fans out
    sim.StartDeal(0, 1000, 10);
    sim.SimTime = GrayboxSim.FirstPlayT;      // 1.50 first feedback triple, 420 = x4
    sim.PlayCard(new int[] { 3, 3, 3, 4 });
    sim.SimTime = PageFrozen.TeachStraightT;  // 2.50 straight, the mult visibly live
    sim.PlayCard(new int[] { 5, 6, 7, 8, 9 });
    sim.SimTime = PageFrozen.TeachFirstWinT;  // 3.50 single closes 1,400 &gt;= 1,000
    sim.PlayCard(new int[] { 10 });
    sim.ResolveDeal();
    sim.SimTime = GrayboxSim.FirstPickT;      // 5.50 first deliberate pick
    sim.PickCharm(2);
    sim.SimTime = PageFrozen.TeachClimaxT;    // 8.00 teaching loop closes
    sim.Caption("Six charms, bombs first, rocket last - the 8192x shot is yours to play.");
}
// ---- save face: lineup / dex / deal progress / adcaps ---------------------
// Service round-trip helpers: the runtime asmdef carries the Sy.Core service
// face (the editor asmdef does not reference it), so the editor gate drives
// the ISaveService round-trip through these two helpers.
public static bool SaveFaceThroughService(SaveFace face)
{
    ServiceLocator.Get&lt;ISaveService&gt;().Save(PageFrozen.SaveSlot, SaveCodec.Compose(face));
    return true;
}
public static string LoadFacePayload()
{
    return ServiceLocator.Get&lt;ISaveService&gt;().Load(PageFrozen.SaveSlot);
}
/// &lt;summary&gt;The persisted face: the charm lineup (pick order preserved), the
/// collected dex (never wiped on loss - zero-punishment law), the deal
/// progress (index + score) and the ad capsules (the T008 AdLedger save.adcaps
/// data face, X995 wall-clock stamps so the rolling window survives restarts).&lt;/summary&gt;
public sealed class SaveFace
{
    public readonly List&lt;int&gt; Lineup = new List&lt;int&gt;();
    public readonly List&lt;int&gt; Dex = new List&lt;int&gt;();
    public int DealIndex;
    public long DealScore;
    public readonly List&lt;AdCapsule&gt; AdCaps = new List&lt;AdCapsule&gt;();
}
/// &lt;summary&gt;
/// Strict save codec (ShareCodec twin law): fixed prefix, fixed section order,
/// strict token grammar, roster-bounded indices, frozen ad categories, and a
```

```
/// canonical compose so Compose(Parse(Compose(x))) == Compose(x) always holds.
/// Format (pure ASCII, zero gambling vocabulary):
///   CRSAVE1|L=0,1,2|D=0,1,2,3|P=2,1400|A=AD_RESCUE@3600:1;AD_DEX_FILL@7200:0
/// The dex composes sorted+deduped (set semantics); the lineup keeps pick
/// order; capsules keep ledger order.
/// &lt;/summary&gt;
public static class SaveCodec
{
    public const string Prefix = "CRSAVE1";
    const int RosterSize = 12;
    public static string Compose(SaveFace face)
    {
        StringBuilder sb = new StringBuilder();
        sb.Append(Prefix);
        sb.Append("|L=");
        AppendInts(sb, face.Lineup);
        sb.Append("|D=");
        List&lt;int&gt; dex = new List&lt;int&gt;(face.Dex);
        dex.Sort();
        for (int i = 1; i &lt; dex.Count; i++) if (dex[i] == dex[i - 1]) { dex.RemoveAt(i); i--; }
        AppendInts(sb, dex);
        sb.Append("|P=").Append(face.DealIndex.ToString(CultureInfo.InvariantCulture))
          .Append(',').Append(face.DealScore.ToString(CultureInfo.InvariantCulture));
        sb.Append("|A=");
        for (int i = 0; i &lt; face.AdCaps.Count; i++)
        {
            if (i &gt; 0) sb.Append(';');
            AdCapsule c = face.AdCaps[i];
            sb.Append(c.Type).Append('@')
              .Append(c.OfferedAt.ToString("R", CultureInfo.InvariantCulture))
              .Append(':').Append(c.Accepted ? '1' : '0');
        }
        return sb.ToString();
    }
    static void AppendInts(StringBuilder sb, List&lt;int&gt; list)
    {
        for (int i = 0; i &lt; list.Count; i++)
        {
            if (i &gt; 0) sb.Append(',');
            sb.Append(list[i].ToString(CultureInfo.InvariantCulture));
        }
    }
    public static bool TryParse(string payload, out SaveFace face)
    {
        face = null;
        if (payload == null) return false;
        string[] sections = payload.Split('|');
        if (sections.Length != 5 || sections[0] != Prefix) return false;
        if (!sections[1].StartsWith("L=", StringComparison.Ordinal) ||
            !sections[2].StartsWith("D=", StringComparison.Ordinal) ||
```

```
                !sections[3].StartsWith("P=", StringComparison.Ordinal) ||
                !sections[4].StartsWith("A=", StringComparison.Ordinal)) return false;
            SaveFace f = new SaveFace();
            if (!ParseIntList(sections[1].Substring(2), f.Lineup, PageFrozen.LineupSlots)) return false;
            if (!ParseIntList(sections[2].Substring(2), f.Dex, int.MaxValue)) return false;
            string p = sections[3].Substring(2);
            int comma = p.IndexOf(',');
            if (comma &lt;= 0) return false;
            if (!int.TryParse(p.Substring(0, comma), NumberStyles.None, CultureInfo.InvariantCulture, out f.DealInde
                f.DealIndex &lt; 0) return false;
            if (!long.TryParse(p.Substring(comma + 1), NumberStyles.None, CultureInfo.InvariantCulture, out f.DealSc
                f.DealScore &lt; 0) return false;
            string a = sections[4].Substring(2);
            if (a.Length &gt; 0)
            {
                string[] caps = a.Split(';');
                for (int i = 0; i &lt; caps.Length; i++)
                {
                    string cap = caps[i];
                    int at = cap.IndexOf('@');
                    int colon = cap.LastIndexOf(':');
                    if (at &lt;= 0 || colon &lt;= at + 1 || colon != cap.Length - 2) return false;
                    string type = cap.Substring(0, at);
                    if (!AdFrozen.IsCategory(type)) return false;   // frozen categories only
                    string flag = cap.Substring(colon + 1);
                    if (flag != "0" &amp;&amp; flag != "1") return false;
                    if (!double.TryParse(cap.Substring(at + 1, colon - at - 1), NumberStyles.Float,
                                         CultureInfo.InvariantCulture, out double stamp) || stamp &lt; 0) return fal
                    f.AdCaps.Add(new AdCapsule { Type = type, OfferedAt = stamp, Accepted = flag == "1" });
                }
            }
            face = f;
            return true;
        }
        static bool ParseIntList(string s, List&lt;int&gt; into, int maxCount)
        {
            if (s.Length == 0) return true;
            string[] parts = s.Split(',');
            if (parts.Length &gt; maxCount) return false;
            for (int i = 0; i &lt; parts.Length; i++)
            {
                if (!int.TryParse(parts[i], NumberStyles.None, CultureInfo.InvariantCulture, out int v)) return fals
                if (v &lt; 0 || v &gt;= RosterSize) return false;   // roster-bounded, 0..11
                into.Add(v);
            }
            return true;
        }
    }
}
}
```

```
// PagesHost.cs
// PagesHost.cs - S1 T009 procedural UGUI pages host for CardRogue (X027 paradigm:
// zero prefabs, zero scene serialization - the main scene only carries the camera and
// the EventSystem; every page is built in code at runtime).
// Four frozen pages (PageFrozen registry): S99_Boot splash, S01_Table (felt table bg
// + hand strip + six lineup slots + target banner + HUD entries + the rescue popup
// slot), S02_Codex twelve-grid wall, S03_Share (share frame + lineup code label).
// U184 per-canvas law: every page canvas carries a CanvasScaler locked to
// ScaleWithScreenSize at the frozen 1080x1920 portrait reference plus a
// GraphicRaycaster. The ProjectSetup placeholder canvas (the "S23_Main Main" label
// shell) is absorbed on build - placeholder shell law, zero stray canvas left behind.
// Teach pacing walks the frozen PagesGate beat table (wall time); captions resolve
// through LocaleService by the beat CaptionKeys (T010 i18n externalization - zero
// hardcoded copy in the view face) and every drive step drains the real-path audio
// hooks (T010 wiring: BGM loop seat + the same-tick feedback trio). Save wiring
// mirrors the sim + ad ledger into the SaveFace and lands it
// through ISaveService (the save.adcaps data face contracted in T008 AdGate).
using System.Collections;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
namespace Sy.CardRogue
{
    public sealed class PagesHost : MonoBehaviour
    {
        static Font _font;
        readonly Dictionary&lt;string, GameObject&gt; _pages = new Dictionary&lt;string, GameObject&gt;();
        GrayboxSim _sim;
        AdGate _adGate;
        public static Font SharedFont
        {
            get
            {
                if (_font == null) _font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
                return _font;
            }
        }
        /// &lt;summary&gt;Display copy resolver: the section-6.3 closed key set through
        /// LocaleService (T010 law: zero hardcoded copy in the view face).&lt;/summary&gt;
        static string L(string key) { return LocaleService.Get(key); }
        /// &lt;summary&gt;Page-name technical token: the frozen registry name minus its
        /// "SXX_" prefix. HUD entry / page-heading labels are registry-derived tokens
        /// (page names, not localized copy - the section-6.3 key set is closed at 35
        /// and the Latin token face rides the X996 En-L0 default).&lt;/summary&gt;
        static string PageToken(string pageName)
        {
            int cut = pageName.IndexOf('_');
            return cut &gt;= 0 &amp;&amp; cut + 1 &lt; pageName.Length ? pageName.Substring(cut + 1) : pageName;
```

```
}
/// &lt;summary&gt;Boot wiring face: find-or-create the host in the freshly loaded
/// main scene (called by BootRunner after the scene switch; idempotent).&lt;/summary&gt;
public static PagesHost EnsureInScene()
{
    PagesHost existing = FindObjectOfType&lt;PagesHost&gt;();
    if (existing != null) return existing;
    var go = new GameObject("PagesHost");
    return go.AddComponent&lt;PagesHost&gt;();
}
void Awake()
{
    BuildAll();
}
IEnumerator Start()
{
    // splash beats: hold the boot page, then hand the table the teach loop
    yield return new WaitForSeconds(0.8f);
    if (_pages.ContainsKey(PageFrozen.Names[0])) _pages[PageFrozen.Names[0]].SetActive(false);
    if (_pages.ContainsKey(PageFrozen.Names[1])) _pages[PageFrozen.Names[1]].SetActive(true);
    yield return TeachLoop();
}
// ==== build face =====================================================
public void BuildAll()
{
    AbsorbPlaceholder();
    EnsureEventSystem();
    for (int i = 0; i &lt; PageFrozen.Names.Length; i++)
    {
        string name = PageFrozen.Names[i];
        if (_pages.ContainsKey(name)) continue;
        _pages[name] = BuildPage(name);
    }
    BuildBootContent();
    BuildTableContent();
    BuildCodexContent();
    BuildShareContent();
    _pages[PageFrozen.Names[0]].SetActive(true);   // splash first
    _pages[PageFrozen.Names[1]].SetActive(true);   // table under the splash
    _pages[PageFrozen.Names[2]].SetActive(false);  // codex overlay
    _pages[PageFrozen.Names[3]].SetActive(false);  // share overlay
}
/// &lt;summary&gt;Placeholder shell law: the ProjectSetup-generated root "Canvas"
/// (with its "S23_Main Main" label) is a skeleton face - absorb it so the
/// built pages are the only canvases (U184: zero stray canvas without scaler).&lt;/summary&gt;
void AbsorbPlaceholder()
{
    Canvas[] roots = FindObjectsOfType&lt;Canvas&gt;();
    for (int i = 0; i &lt; roots.Length; i++)
    {
```

```
            Canvas c = roots[i];
            if (c.transform.parent != null) continue;
            if (PageFrozen.IsPage(c.name)) continue;
            if (c.name == "Canvas")
            {
                if (Application.isPlaying) Destroy(c.gameObject);
                else DestroyImmediate(c.gameObject);
            }
        }
    }
    void EnsureEventSystem()
    {
        if (FindObjectOfType&lt;EventSystem&gt;() != null) return;
        var esGo = new GameObject("EventSystem");
        esGo.AddComponent&lt;EventSystem&gt;();
        esGo.AddComponent&lt;StandaloneInputModule&gt;();
    }
    GameObject BuildPage(string name)
    {
        var go = new GameObject(name);
        Canvas canvas = go.AddComponent&lt;Canvas&gt;();
        canvas.renderMode = RenderMode.ScreenSpaceOverlay;
        // U184 per-canvas law: scaler locked to ScaleWithScreenSize at the frozen
        // portrait reference + raycaster on every page canvas.
        CanvasScaler scaler = go.AddComponent&lt;CanvasScaler&gt;();
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        scaler.referenceResolution = new Vector2(PageFrozen.RefWidth, PageFrozen.RefHeight);
        scaler.matchWidthOrHeight = 0.5f;
        go.AddComponent&lt;GraphicRaycaster&gt;();
        return go;
    }
    // ---- shared widget helpers ----------------------------------------
    static RectTransform NewRect(Transform parent, string name)
    {
        var go = new GameObject(name, typeof(RectTransform));
        go.transform.SetParent(parent, false);
        return go.GetComponent&lt;RectTransform&gt;();
    }
    static Image NewImage(Transform parent, string name, Color32 color, float w, float h, float x, float y)
    {
        RectTransform rt = NewRect(parent, name);
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = new Vector2(x, y);
        Image img = rt.gameObject.AddComponent&lt;Image&gt;();
        img.color = color;
        return img;
    }
    static Text NewText(Transform parent, string name, string str, int size, float w, float h, float x, float y, Col
    {
        RectTransform rt = NewRect(parent, name);
```

```
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = new Vector2(x, y);
    Text t = rt.gameObject.AddComponent&lt;Text&gt;();
    t.font = SharedFont;
    t.text = str;
    t.fontSize = size;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = color;
    return t;
}
static readonly Color32 Felt = new Color32(0x26, 0x9B, 0x5E, 0xFF);        // bright jade felt (S0 frozen accent
static readonly Color32 FeltDark = new Color32(0x1E, 0x7A, 0x49, 0xFF);
static readonly Color32 Gold = new Color32(0xFF, 0xD7, 0x00, 0xFF);        // blast gold - accent only, never th
static readonly Color32 Ink = new Color32(0x1A, 0x2B, 0x22, 0xFF);
static readonly Color32 Paper = new Color32(0xFA, 0xF6, 0xE9, 0xFF);
static readonly Color32 Slot = new Color32(0x14, 0x4D, 0x30, 0xFF);
// ---- S99_Boot: splash with the title + slogan -----------------------
void BuildBootContent()
{
    Transform page = _pages["S99_Boot"].transform;
    NewImage(page, "SplashBg", FeltDark, 0, 0, 0, 0).rectTransform.StretchFull();
    NewText(page, "PageTitleLabel", L("ui_title"), 96, 900, 160, 0, 260, Paper);
    NewText(page, "PageSloganLabel",
            L("ui_slogan"), 40, 1000, 80, 0, 100, Paper);
}
// ---- S01_Table: the felt battlefield --------------------------------
void BuildTableContent()
{
    Transform page = _pages["S01_Table"].transform;
    NewImage(page, "FeltTableBg", Felt, 0, 0, 0, 0).rectTransform.StretchFull();
    NewImage(page, "TargetBanner", Ink, 1040, 120, 0, -100);
    Transform banner = page.Find("TargetBanner");
    NewText(banner, "TargetBannerText", L("target_label"), 44, 1000, 100, 0, 0, Gold);
    NewImage(page, "HudCodexEntry", Paper, 260, 90, -390, -250);
    Transform codex = page.Find("HudCodexEntry");
    NewText(codex, "HudCodexText", PageToken(PageFrozen.Names[2]), 36, 240, 80, 0, 0, Ink);
    NewImage(page, "HudShareEntry", Paper, 260, 90, 390, -250);
    Transform share = page.Find("HudShareEntry");
    NewText(share, "HudShareText", PageToken(PageFrozen.Names[3]), 36, 240, 80, 0, 0, Ink);
    NewImage(page, "HandStrip", Slot, 1040, 340, 0, -640);
    for (int i = 0; i &lt; PageFrozen.LineupSlots; i++)
        NewImage(page, "LineupSlot_" + i, Slot, 140, 180, -450 + i * 180, -260);
    NewImage(page, "RescuePopup", Ink, 820, 900, 0, 0);
    Transform popup = page.Find("RescuePopup");
    NewText(popup, "RescuePopupText",
            L("rescue_offer_title"), 36, 760, 120, 0, 320, Paper);
    NewText(popup, "RescuePopupNote", L("rescue_skip"), 30, 760, 80, 0, -380, Gold);
    page.Find("RescuePopup").gameObject.SetActive(false);   // popup slot: parked until a near-miss
    // T013 c2 event popup slot: the comedy event copy slot (four beat
    // positions x three style families). The frame seat carries the AR-11
```

```
    // sprite when it lands (null-safe, armed-silent before arrival - a plain
    // tinted panel meanwhile); the badge is the family technical token.
    NewImage(page, "EventPopup", Ink, 820, 640, 0, 40);
    Transform eventPopup = page.Find("EventPopup");
    NewImage(eventPopup, "EventPopupFrame", EventPopupRuntime.TintOf("news"), 740, 140, 0, 210);
    NewText(eventPopup, "EventPopupText", "", 30, 740, 380, 0, -40, Paper);
    NewText(eventPopup, "EventPopupBadge", "", 26, 240, 56, -260, 278, EventPopupRuntime.TintNews);
    page.Find("EventPopup").gameObject.SetActive(false);   // parked until a beat fires
}
// ---- S02_Codex: the twelve-grid wall --------------------------------
void BuildCodexContent()
{
    Transform page = _pages["S02_Codex"].transform;
    NewImage(page, "CodexBg", FeltDark, 0, 0, 0, 0).rectTransform.StretchFull();
    NewText(page, "CodexTitleLabel", PageToken(PageFrozen.Names[2]), 64, 900, 120, 0, -780, Paper);
    for (int i = 0; i &lt; PageFrozen.CodexCells; i++)
    {
        int col = i % 3, row = i / 3;
        NewImage(page, "CodexCell_" + i, Slot, 280, 280, -320 + col * 320, 520 - row * 320);
    }
}
// ---- S03_Share: the lineup card -------------------------------------
void BuildShareContent()
{
    Transform page = _pages["S03_Share"].transform;
    NewImage(page, "ShareDim", Ink, 0, 0, 0, 0).rectTransform.StretchFull();
    NewImage(page, "ShareFrame", Paper, 700, 1200, 0, 0);
    Transform frame = page.Find("ShareFrame");
    NewText(frame, "ShareCodeLabel", ShareCodec.Prefix, 44, 620, 90, 0, 480, Ink);
    for (int i = 0; i &lt; PageFrozen.LineupSlots; i++)
        NewImage(frame, "ShareSlot_" + i, Felt, 150, 200, -360 + i * 144, 0);
}
// ==== runtime faces ================================================
/// &lt;summary&gt;Teach pacing: walk the frozen beat table in wall time and show the
/// caption mirrors on the table caption label; run the same drive on the real
/// sim (the E4 Stage1 timeline) and land the save face at the end.&lt;/summary&gt;
IEnumerator TeachLoop()
{
    _sim = new GrayboxSim();
    _adGate = new AdGate();
    Text caption = null;
    Transform table = _pages.ContainsKey("S01_Table") ? _pages["S01_Table"].transform : null;
    if (table != null)
    {
        NewText(table, "TeachCaptionLabel", "", 40, 1000, 80, 0, -120, Paper);
        caption = table.Find("TeachCaptionLabel").GetComponent&lt;Text&gt;();
    }
    AudioRuntime.PlayBgm();   // AU-01 table BGM loop seat (S01 global face, T010 wiring)
    double last = 0.0;
    int drained = 0;          // audio hook drain index into the sim event stream
```

```
    int popupDrained = 0;     // event popup drain index (T013 c2 four-beat wiring)
    for (int i = 0; i &lt; PagesGate.TeachBeats.Length; i++)
    {
        TeachBeat beat = PagesGate.TeachBeats[i];
        double wait = beat.T - last;
        if (wait &gt; 0) yield return new WaitForSeconds((float)wait);
        last = beat.T;
        RunTeachBeat(_sim, i);
        AudioRuntime.Drain(_sim, drained);   // real-path audio hooks (T010 law)
        drained = _sim.Events.Count;
        EventPopupRuntime.Drain(_sim, popupDrained);   // four-beat event popups (T013 c2)
        popupDrained = _sim.Events.Count;
        ApplyEventPopup(table);
        if (caption != null &amp;&amp; beat.CaptionKey != null) caption.text = L(beat.CaptionKey);
        if (_sim.RescueOffers &gt; 0 &amp;&amp; table != null &amp;&amp; !table.Find("RescuePopup").gameObject.a
            table.Find("RescuePopup").gameObject.SetActive(true);   // near-miss face: popup slot wakes
    }
    CaptureAndSave();
}
/// &lt;summary&gt;T013 c2: apply the latest event popup fired by the runtime drain
/// (family rotation + AR-11 frame seat, armed-silent before the art lands -
/// a plain tinted panel meanwhile, zero crash, zero tofu). Zero hardcoded
/// copy: the label resolves through LocaleService by the composed key.&lt;/summary&gt;
void ApplyEventPopup(Transform table)
{
    string key = EventPopupRuntime.LastShownKey;
    if (key == null || table == null) return;
    Transform popup = table.Find("EventPopup");
    if (popup == null) return;
    Transform frame = popup.Find("EventPopupFrame");
    Transform labelT = popup.Find("EventPopupText");
    Transform badgeT = popup.Find("EventPopupBadge");
    if (labelT != null) labelT.GetComponent&lt;Text&gt;().text = L(key);
    string family = EventPopupRuntime.FamilyOf(key);
    if (badgeT != null) badgeT.GetComponent&lt;Text&gt;().text = EventPopupRuntime.BadgeOf(family);
    if (frame != null)
    {
        Image img = frame.GetComponent&lt;Image&gt;();
        img.sprite = EventPopupRuntime.FrameSprite;      // null-safe: AR-11 pending stays a tinted panel
        img.color = EventPopupRuntime.TintOf(family);    // one frame, three families: programmatic tint
        if (badgeT != null) badgeT.GetComponent&lt;Text&gt;().color = EventPopupRuntime.TintOf(family);
    }
    popup.gameObject.SetActive(true);
}
/// &lt;summary&gt;One beat of the frozen E4 timeline on the real sim (the same
/// sequence PagesGate.RunTeachDrive replays in logical time).&lt;/summary&gt;
public static void RunTeachBeat(GrayboxSim sim, int beatIndex)
{
    switch (beatIndex)
    {
```

```
                case 0: sim.SimTime = GrayboxSim.CaptionT; sim.Caption("This run: 8192x."); break;
                case 1: sim.SimTime = GrayboxSim.StarterCharmT; sim.StartRun(0); break;
                case 2: sim.SimTime = GrayboxSim.DealFanT; sim.StartDeal(0, 1000, 10); break;
                case 3: sim.SimTime = GrayboxSim.FirstPlayT; sim.PlayCard(new int[] { 3, 3, 3, 4 }); break;
                case 4: sim.SimTime = PageFrozen.TeachStraightT; sim.PlayCard(new int[] { 5, 6, 7, 8, 9 }); break;
                case 5: sim.SimTime = PageFrozen.TeachFirstWinT; sim.PlayCard(new int[] { 10 }); sim.ResolveDeal(); brea
                case 6: sim.SimTime = GrayboxSim.FirstPickT; sim.PickCharm(2); break;
                case 7: sim.SimTime = PageFrozen.TeachClimaxT;
                        sim.Caption("Six charms, bombs first, rocket last - the 8192x shot is yours to play."); break;
            }
        }
        /// &lt;summary&gt;Save wiring: mirror the sim (lineup / dex / deal progress) and the
        /// ad ledger (adcaps, X995 wall stamps) into the SaveFace, then land it through
        /// ISaveService. The T008 contract wired here: AdLedger IS the save.adcaps face.&lt;/summary&gt;
        public void CaptureAndSave()
        {
            var face = new PagesGate.SaveFace();
            if (_sim != null)
            {
                face.Lineup.AddRange(_sim.Lineup);
                face.Dex.AddRange(_sim.Dex);
                face.DealIndex = _sim.DealsStarted &gt; 0 ? _sim.DealsStarted - 1 : 0;
                face.DealScore = _sim.DealScore;
            }
            if (_adGate != null)
                for (int i = 0; i &lt; _adGate.Ledger.Capsules.Count; i++)
                    face.AdCaps.Add(new AdCapsule
                    {
                        Type = _adGate.Ledger.Capsules[i].Type,
                        OfferedAt = _adGate.Ledger.Capsules[i].OfferedAt,
                        Accepted = _adGate.Ledger.Capsules[i].Accepted
                    });
            ServiceLocator.Get&lt;ISaveService&gt;().Save(PageFrozen.SaveSlot, PagesGate.SaveCodec.Compose(face));
        }
    }
    /// &lt;summary&gt;Full-stretch helper for backdrop rects (anchor both corners, zero offsets).&lt;/summary&gt;
    public static class RectTransformExtensions
    {
        public static RectTransform StretchFull(this RectTransform rt)
        {
            rt.anchorMin = Vector2.zero;
            rt.anchorMax = Vector2.one;
            rt.offsetMin = Vector2.zero;
            rt.offsetMax = Vector2.zero;
            return rt;
        }
    }
}
// ShareCodec.cs
// ShareCodec.cs - S1 T007 X032 offline share codec for CardRogue (compose/decode round-trip).
```

```
// The compose face mirrors the frozen GrayboxSim.ShareCard() encoding byte-for-byte:
// "CR1-" prefix + one uppercase hex digit per collected dex position, ascending roster
// order (dex-position encoding, needs-doc 6.2 / E4 assertion 11). Decode is the strict
// offline replay face: a well-formed code alone rebuilds the charm build with zero
// server and zero guesswork (X032 offline form, red line 12). Score-form words and
// gambling imagery are banned from every share payload face by construction
// (B904 guardrails #2 / #5). No UnityEngine dependency - pure C#.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.CardRogue
{
    public static class ShareCodec
    {
        public const string Prefix = "CR1-";
        // Compose: collected dex positions -&gt; "CR1-" + ascending uppercase hex digits.
        // Byte-for-byte mirror of the frozen GrayboxSim.ShareCard() code face.
        public static string Compose(IEnumerable&lt;int&gt; dexPositions)
        {
            if (dexPositions == null) return null;
            bool[] has = new bool[GrayboxSim.CharmRoster.Length];
            foreach (int p in dexPositions)
            {
                if (p &lt; 0 || p &gt;= has.Length) return null;   // off-roster input = malformed
                has[p] = true;
            }
            StringBuilder sb = new StringBuilder(Prefix);
            for (int i = 0; i &lt; has.Length; i++)
                if (has[i]) sb.Append(i.ToString("X", CultureInfo.InvariantCulture));
            return sb.ToString();
        }
        // TryDecode: strict offline replay. Accepts only the canonical compose form:
        // the "CR1-" prefix plus ascending, unique, in-roster uppercase hex digits.
        public static bool TryDecode(string code, List&lt;int&gt; dexPositions)
        {
            if (dexPositions == null) return false;
            dexPositions.Clear();
            if (string.IsNullOrEmpty(code)) return false;
            if (!code.StartsWith(Prefix, StringComparison.Ordinal)) return false;
            string hex = code.Substring(Prefix.Length);
            if (hex.Length == 0) return false;
            int prev = -1;
            for (int i = 0; i &lt; hex.Length; i++)
            {
                int v = HexVal(hex[i]);
                if (v &lt; 0) return false;                                  // non-hex / lowercase face
                if (v &gt;= GrayboxSim.CharmRoster.Length) return false;    // off-roster position
                if (v &lt;= prev) return false;                              // duplicate / non-ascending
                dexPositions.Add(v);
```

```
                prev = v;
            }
            return true;
        }
        static int HexVal(char c)
        {
            if (c &gt;= '0' &amp;&amp; c &lt;= '9') return c - '0';
            if (c &gt;= 'A' &amp;&amp; c &lt;= 'F') return c - 'A' + 10;
            return -1;
        }
    }
}
```